

1. Parsing Table conflict in LL (1)

28/50

In an LL(1) parser, a parsing table conflict occurs when more than one production is entered into the same table cell. This usually happens when the grammar has overlapping FIRST SETS or when FIRST and FOLLOW sets create conflicts.

Because of this conflict, the parser cannot uniquely decide which production to select. Hence, determine LL(1) parsing is not possible.

Decision: The grammar should be modified using left recursion eliminating or left factor. This ensures that each parsing table cell contains only one production and makes the grammar suitable for LL(1) parsing.

2) Incorrect parse Tree Generation

An Syntax Analyses generates a parse tree based on the grammar rules. If the grammar does not properly define associativity, an exp such as $id + id - id$ may produce an incorrect parse tree.

For example.

$$id + (id - id)$$

instead of the correct left associative form:

$$(id + id) - id$$

this can lead to incorrect evaluation

Decision:

The grammar should be rewritten to enforce left associativity. or appropriate precedence and associativity rules should be applied during Parsing

3 Invalid Expression Handling

For input $a + * b$, all tokens are individually valid, but their arrangement is invalid according to the grammar. After the $+$ operator, the parser expects an operand, but it receives another $*$ operator

In predictive parsing, no suitable production can be selected. In shift-reduce parsing, neither can a valid shift nor reduce action

Decision:

The parser should use error recovery techniques such as panic-mode recovery or phrase-level recovery. These allow the error to be detected clearly and parsing to continue when possible.

4) Ambiguity in conditional statements:

the grammar is:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow a$

This grammar creates the well-known dangling else problem. When an else is encountered, the parser cannot determine whether it belongs to the nearest if or an earlier if.

Decision: The grammar should be modified to separate matched and unmatched statements. Alternatively, the parser can follow the standard rule that an else is associated with the nearest unmatched if. This removes the ambiguity.

5) Operator precedence conflict

The grammar is:

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

For input $id + id * id$, the grammar is ambiguous because it does not specify operator precedence. The parser may incorrectly treat $+$ and $*$ with the same precedence.

However, multiplication ($*$) should have higher precedence than addition ($+$). Therefore, the correct grouping is $id + (id * id)$.

Decision: The grammar should be rewritten as:

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow id$$

This grammar gives $*$ higher precedence than $+$.